

Uçuş Kontrol Yazılımı

Gömülü gerçek zamanlı yazılımın mimarisi, kaynak kodu ve satır düzeyinde gerekçelendirmesi

«sketch» verici	«sketch» alici	«sketch» arac	«sketch» arac
verici_v2_kumanda.ino el kumandası	alici_v3_ucus_kontrol.ino uçuş kontrolcüsü	arac_esc_kalibrasyon.ino ESC uç nokta	arac_motor_test.ino yön doğrulama

UÇUŞ YAZILIMLARI

ARAÇ / DEVREYE ALMA YAZILIMLARI

Doküman kodu	SWD-QC-002
Üst doküman	SDD-QC-001 — Sistem Tasarım Dokümanı
Dil / araç zinciri	C++ (Arduino framework), ESP32 Arduino Core
Hedef	ESP32-WROOM-32, 240 MHz, çift çekirdek
Kontrol döngüsü	250 Hz · motor darbesi 50 Hz (RMT)
Modül sayısı	2 uçuş yazılımı + 2 araç yazılımı
Toplam satır	≈ 900 satır (yorumlar dâhil)
Bağımlılıklar	RF24, MPU6050_light, Wire, SPI (ESP32Servo yok)

1. Yazılım Mimarisi

Yazılım iki uçuş modülü ve iki araç modülünden oluşur. Uçuş modülleri kalıcı olarak sahada çalışır; araç modülleri yalnızca devreye alma sırasında yüklenir ve doğrulama tamamlandığında kartlarda kalmaz.

Modül	Hedef düğüm	Rol	Yaşam döngüsü
verici_v2_kumanda	Verici ESP32	Pilot girdisini okur, ölçekler ve telsizle yayınlar	Kalıcı
alici_v3_ucus_kontrol	Alıcı ESP32	Komut alır, aç kestirir, PID uygular, motorları sürer	Kalıcı
arac_esc_kalibrasyon	Alıcı ESP32	Dört ESC'ye ortak gaz aralığı öğretir	Bir kerelik
arac_motor_test	Alıcı ESP32	Motorları tek tek çalıştırıp dönüş yönünü doğrular	Devreye alma

1.1 Katmanlı yapı

Uçuş kontrol yazılımı, her katmanın yalnızca bir alttakine bağımlı olduğu düz bir katman yapısıyla kurgulanmıştır. Bu, bir katmanın bağımsız olarak test edilebilmesini sağlar — nitekim geliştirme sırası da tam olarak bu katmanları izlemiştir.



Şekil 1.1 — Yazılım katmanları ve doğrulama sırası. Her katman, bir üsttekine geçilmeden önce bağımsız olarak doğrulanmıştır.

2. Telsiz Haberleşme Protokolü

Haberleşme, sabit uzunlukta bir C yapısının (struct) doğrudan bayt dizisi olarak gönderilmesine dayanır. Bu yaklaşım, serileştirme maliyetini sıfırlar ve gecikmeyi öngörülebilir kılar; karşılığında **iki taraftaki yapı tanımının birebir aynı olması zorunludur**.

2.1 Çerçeve yapısı

Alan	Tip	Bayt	Aralık	Anlam
kanal[0]	uint16_t	0-1	1000-2000	Gaz komutu (µs)
kanal[1]	uint16_t	2-3	1000-2000	Yaw komutu (µs)
kanal[2]	uint16_t	4-5	1000-2000	Pitch komutu (µs)
kanal[3]	uint16_t	6-7	1000-2000	Roll komutu (µs)
sayac	uint16_t	8-9	0-65535	Paket sırası (kayıp tespiti)

Toplam çerçeve boyutu **10 bayt**'tır; nRF24'ün 32 baytlık sabit yük kapasitesinin çok altındadır. 50 Hz çerçeve hızında ham veri yükü yalnızca 4 kbit/s'dir — seçilen 250 kbps veri hızının %2'si. Bu geniş marj, düşük veri hızının menzil avantajından ödün vermeden kullanılabilmesini sağlar.

2.2 Radyo parametreleri ve gerekçeleri

Parametre	Varsayılan	Seçilen	Gerekçe
Veri hızı	1 Mbps	250 kbps	Alıcı duyarlılığı artar, menzil yaklaşık iki katına çıkar; veri yükü zaten çok düşük
Kanal	76 (2476 MHz)	100 (2500 MHz)	2.4 GHz WiFi bandının yoğun bölgesinden uzaklaşma
Çıkış gücü	PA_MAX	PA_LOW (test) / PA_MAX (uçuş)	Modüller çok yakinken yüksek güç alıcıyı doyuma sokabiliyor
Adres	0xE7E7E7E7E7	"DRN01"	Varsayılan adres yaygındır; çakışma riskini azaltmak için özel adres
Otomatik ACK	Açık	Açık	Gönderim başarısı <code>radio.write()</code> dönüş değerinden anlaşılır — bağlantı tespiti için kullanılır
CRC	16 bit	16 bit	Çerçeve bütünlüğü doğrulaması

2.3 Çerçeve doğrulama ve kayıp sayımı

Alıcı, gelen her çerçeveyi işlemeyen önce iki aşamalı bir süzgeçten geçirir. Bu mekanizma, geliştirme sırasında gözlenen gerçek bir arızaya (D-03) yanıt olarak eklenmiştir.

- Aralık süzgeci.** Dört kanalın tümü 950-2050 µs bandında olmalıdır. Bant dışı bir değer, çerçevenin tamamının reddedilmesine yol açar — kısmi kabul yapılmaz, çünkü bozuk bir çerçevede tek bir alanın sağlam olduğu varsayılmaz.
- Sayaç sürekliliği.** Ardışık çerçevelerin sayaç farkı 1'den büyükse aradaki fark kayıp paket olarak birikir. Bu sayaç, telemetri üzerinden menzil ve girişim analizinde kullanılır.

Neden bu süzgeç kritik

Verici beslemesiz kaldığında alıcının SPI tamponu tanımsız içerik döndürür; bu, kanal alanlarında 0 veya 65535 gibi uç değerler olarak görünür. Süzgeç olmasaydı bu değerler doğrudan motor karıştırma matrisine girer ve dört motora aynı anda sıçrama komutu üretirdi. Süzgeç, çerçeve düzeyinde bir *bütünlük sözleşmesi* uygular.

3. Verici Yazılımı — El Kumandası

Verici modülü üç işi sırayla yapar: analog eksenleri okur, bunları standart 1000–2000 μ s bandına ölçekler ve çerçeveyi yayınlar. Basit görünen bu zincirin içinde, gerçek donanımın dayattığı üç tasarım kararı gizlidir.

3.1 Pin tahsisi ve donanım kısıtı

verici_v2_kumanda.ino · satır 26–34

```
26  /* ----- PINLER */
27  #define PIN_NRF_CE  4
28  #define PIN_NRF_CSN  5
29  #define PIN_GAZ      32    // sol joystick - dikey   (ADC1)
30  #define PIN_YAW      33    // sol joystick - yatay   (ADC1)
31  #define PIN_PITCH    34    // sag joystick - dikey   (ADC1)
32  #define PIN_ROLL     35    // sag joystick - yatay   (ADC1)
33  // NOT: Dört eksen de ADC1 pinlerindedir. ADC2 telsizle aynı anda
34  //      kullanılamaz (ESP32 donanım kisiti).
```

Dört eksenin de **ADC1** bloğuna (GPIO 32–39) yerleştirilmesi bir tercih değil, zorunluluktur: ESP32'de ADC2 çevre birimi Wi-Fi/telsiz yığını etkinken kullanılamaz. Ayrıca 38 pinli DevKit sürümü seçilmiştir, çünkü 30 pinli sürüm GPIO 34–39 hatlarını dışarı vermez.

3.2 Kalibrasyon sabitleri

kalibrasyon sabitleri · satır 36–46

```
36  /* ----- KALIBRASYON (OLCUM SONUCU) */
37  // Her joystick serbest birakildiginda okunan ham ADC degeri.
38  // Fabrikasyon toleransi nedeniyle 2048 degil, eksen basina farklidir.
39  const uint16_t MERKEZ_GAZ  = 1850;
40  const uint16_t MERKEZ_YAW  = 1900;
41  const uint16_t MERKEZ_PITCH = 1860;
42  const uint16_t MERKEZ_ROLL = 1880;
43
44  const uint16_t OLU_BOLGE    = 80;    // +/- ADC birimi - titremeyi susturur
45  const uint16_t GAZ_OLU_BOLGE = 200;  // gaz icin daha genis bant
46  const float    GAZ_HIZI     = 12.0;  // us / dongu - gaz degisim hizi
```

Joystick potansiyometreleri serbest konumda kuramsal orta değer olan 2048'i vermez; ölçülen merkezler eksen başına 1850–1900 bandındadır. Merkez değerleri koda sabit olarak girilmiş ve ölçekleme merkezin iki yanında ayrı ayrı yapılmıştır. Böylece fiziksel kaçıklık ne olursa olsun çıkış tam olarak 1500 μ s'de merkezlenir.

Ölü bölge başlangıçta ± 60 ADC birimi seçilmişti. Seri port kayıtlarında yaw kanalında 1500–1505 arası artık titreşim gözlemlendi; bant ± 80 'e genişletilerek titreşim tümüyle giderildi. Bu, ölçüme dayalı bir düzeltmenin somut örneğidir — titreşim doğrudan motorlara iletileseydi araç yerinde dururken sürekli düzeltme yapmaya çalışırdı.

3.3 Kanal ölçekleme fonksiyonu

kanalOku() · satır 76–91

```
76  Paket veri;
77
78  float    gaz      = 1000.0; // kilitli gaz - guc kesilene kadar korunur
79  uint32_t cerceveSon = 0;
80  uint32_t yazSon    = 0;
81  uint16_t gonderilen = 0;    // son pencerede gonderilen cerceve sayisi
82  uint16_t basarili   = 0;    // son pencerede ACK donen cerceve sayisi
83  uint8_t kalite      = 0;    // baglanti kalitesi, yuzde
84
85  /* =====
```

```

86 * kanal0ku - ham ADC degerini 1000-2000 us bandina ceviriir
87 *
88 * Merkezin iki yaninda AYRI olceklemeye yapilir. Boylece merkez degeri
89 * 2048'den sapmis olsa bile cikis tam olarak 1500'de merkezlenir.
90 * Tek bir map() kullanilsaydi merkez kacikligi dogrudan cikisa tasinirdi.
91 * ===== */

```

Fonksiyon üç dallıdır: merkezin üstü, merkezin altı ve ölü bölge. İki ayrı map() çağırısı kullanılmasının nedeni, merkezin gerçek orta noktada olmamasıdır — tek bir ölçekleme, merkez kaçıklığını çıkışa taşırdı. ters parametresi, motor yerleşimine göre eksen yönünün yazılımda çevrilebilmesini sağlar; kablo değiştirmeye gerek bırakmaz.

3.4 Kilitli gaz mantığı

Bu blok, projenin en öğretici tasarım kararlarından birini içerir. Kumanda modülünde tek bir merkezleme yayı iki eksenı birden merkezliyordu; gaz ekseninin bıraktığı yerde kalması için yay sökölünce yaw eksenı de merkezlemesini kaybetti. Mekanik çözüm geri alındı ve sorun yazılım katmanında çözüldü.

kilitli gaz + karesel tepki · satır 103–137

```

103
104   cikis = constrain(cikis, 1000, 2000);
105   if (ters) cikis = 3000 - cikis;      // eksenı yazılımda tersine ceviri
106   return (uint16_t)cikis;
107 }
108
109 /* =====
110 * gaziGuncelle - kilitli gaz mantigi
111 *
112 * Joystick modulunde tek merkezleme yayı iki eksenı birden kontrol
113 * ediyordu; gaz icin yay sokulduğunde yaw eksenı de merkezlemesini
114 * kaybetti. Cozum: yay yerinde bırakildi, gaz yazılımda kilitlendi.
115 *   kol yukari -> gaz artar
116 *   kol serbest -> gaz son degerinde kalir
117 *   kol TAM asagi -> ACIL KESME, gaz anında 1000
118 * ===== */
119 void gaziGuncelle() {
120   int hamGaz = analogRead(PIN_GAZ);
121
122   if (hamGaz > MERKEZ_GAZ + GAZ_OLU_BOLGE) {
123     /* --- YUKARI: gaz artar, karesel tepki egrisiyle.
124      Karesel egrı sayesinde kolun az itildiğı bolgede degisim cok
125      yavas, tam itildiğinde hizlidir. Pratik karsiligi, aracın havada
126      asılı kaldığı gaz noktası çevresinde ince ayar yapılabilmesidir. */
127     float oran = (float)(hamGaz - MERKEZ_GAZ - GAZ_OLU_BOLGE)
128                 / (4095.0 - MERKEZ_GAZ - GAZ_OLU_BOLGE);
129     oran = constrain(orán, 0.0, 1.0);
130     gaz += GAZ_HIZI * oran * oran;
131
132   } else if (hamGaz < MERKEZ_GAZ - GAZ_OLU_BOLGE) {
133     float oran = (float)(MERKEZ_GAZ - GAZ_OLU_BOLGE - hamGaz)
134                 / (MERKEZ_GAZ - GAZ_OLU_BOLGE);
135     oran = constrain(orán, 0.0, 1.0);
136
137     if (oran > ACIL_KES_ESIK) {

```

Kol itildiğı sürece gaz değışkeni artar, serbest bırakıldığında son değeri donar. **Karesel tepki eğrisi** ($\text{oran} * \text{oran}$) ise kolun az itildiğı durumda değışimi çok yavaşlatır, tam itildiğinde hızlandırır. Bunun pratik karşılığı, aracın havada asılı kaldığı gaz noktası çevresinde ince ayar yapılabilmesidir — doğrusal bir eğride bu bölge fazla hassas olurdu.

Emniyet notu

gaz değişkeni setup () içinde her açılışta 1000'e sıfırlanır. Kilitli gaz mantığının kabul edilebilir olmasının koşulu budur: kumanda resetlense bile gaz sıfırdan başlar, önceki değeri hatırlamaz.

4. Alıcı Yazılımı — Uçuş Kontrolcüsü

Alıcı modülü sistemin çekirdeğidir. Telsiz alımı, atalet kestirimi, emniyet mantığı, kontrol yasası ve motor sürüşü tek bir sabit periyotlu döngüde birleşir. Aşağıda modül, mantıksal bloklara ayrılarak tam olarak sunulmuştur.

4.1 Motor numaralandırma ve pin haritası

pin tanımları · satır 47–73

```

47 #define PIN_I2C_SCL 22
48
49 // Motor numaralandirmasi - X konfigurasyonu, USTTEN bakis
50 //
51 //      ON (kirmizi kollar)
52 //      M4 \                / M1
53 //      (CCW)                (CW)
54 //      \      _____ /
55 //      |      |         |
56 //      |      |         |
57 //      /      ----- \
58 //      (CW)                (CCW)
59 //      M3 /                \ M2
60 //      ARKA (siyah kollar)
61 //
62 /* MOTOR PINLERI - v4.1'de DEGISTI, KABLO TASINACAK
63     Eski: 12, 13, 14, 27. Bu pinlerde LEDC PWM acikken nRF24 paket
64     kaybi %60 olculdu; PWM kapatilince kayip sifira dustu.
65     GPIO 12 : strapping pini (MTDI) - acilista flash gerilimini belirler
66     GPIO 13 : HSPI MOSI / JTAG
67     GPIO 14 : HSPI CLK / JTAG
68     Yeni pinler bu ozel islevlerin hicbirine sahip degil ve alicida
69     baska bir sey tarafından kullanilmiyor. */
70 #define PIN_M1 25 // on-sag , CW
71 #define PIN_M2 26 // arka-sag, CCW
72 #define PIN_M3 32 // arka-sol, CW
73 #define PIN_M4 33 // on-sol , CCW

```

Motor numaralandırması yorum bloğundaki şemayla belgelenmiştir. Bu belgeleme kozmetik değildir: karıştırma matrisinin işaretleri doğrudan bu yerleşime bağlıdır ve yanlış eşleştirme, aracın düzeltme yerine ters yönde tepki vermesine yol açar.

4.2 Sistem sabitleri

sabitler · satır 75–98

```

75 /* ----- SABITLER */
76 const byte ADRES[6] = "DRN01";
77 const uint8_t RF_KANAL = 115; // VERICI ILE AYNI OLMALI (2515 MHz)
78
79 /* RF CIKIS GUCU - VERICI ILE AYNI OLMALI
80     RF24_PA_MIN : masa ustü / yakin mesafe testleri <-- SIMDI BURADASIN
81     RF24_PA_LOW : birkac metre
82     RF24_PA_HIGH : orta menzil
83     RF24_PA_MAX : acik alan ucusu
84     DIKKAT: Yakin mesafede yuksek guc alicinin giris katini DOYURUR ve
85     paket kaybi ARTAR. Mesafe acildikca kademeli yükseltilir. */
86 #define RF_GUC RF24_PA_MIN
87 const uint16_t PWM_MIN = 1000; // us - motor durdu
88 const uint16_t PWM_MAX = 2000; // us - tam gaz
89 const uint16_t PWM_ARM = 1100; // us - rolanti (armed, yerde)
90 const uint16_t PWM_TAVAN = 1900; // us - PID icin bas payi
91 const uint16_t GAZ_ESIK = 1050; // altinda "gaz kapali" sayilir
92 const uint16_t BAGLANTI_ZAMAN = 500; // ms - failsafe esigi

```

```

93  const float  DONGU_HZ      = 250.0;    // Hz - kontrol dongusu
94  const uint32_t DONGU_US    = (uint32_t)(1000000.0 / DONGU_HZ);
95
96  // Kumanda komut olcekleri
97  const float MAX_ACI        = 25.0;    // derece - roll/pitch komut siniri
98  const float MAX_YAW_HIZ    = 120.0;    // derece/s - yaw komut siniri

```

İki değer özellikle dikkat ister. PWM_TAVAN 1900 µs olarak seçilmiştir, 2000 değil: PID'nin düzeltme yapabilmesi için tepe gazın altında bir **baş payı** kalmalıdır. Tüm motorlar tavana dayanmışsa kontrolcü artık hiçbir eksenle düzeltme üretemez. DONGU_HZ ise 250 Hz'dir; bu değerden türetilen sabit periyot, PID'nin türev ve integral terimlerinin tutarlılığı için zorunludur.

4.3 Darbe üretimi: neden LEDC değil RMT

ESC'ler 50 Hz'lik bir çerçeve içinde 1000–2000 µs genişliğinde darbe bekler. ESP32'de bunu üretmenin iki yolu vardır: LEDC (PWM birimi) veya RMT. İlk gerçeklemede ESP32Servo kütüphanesi, ardından doğrudan LEDC kullanıldı; her ikisinde de telsizde **%60 paket kaybı** ölçüldü. Frekans ve çözünürlük kombinasyonları denendi:

Yapılandırma	Paket kaybı	ESC senkronu
Minimal test (yalnızca telsiz, PWM yok)	%0	—
ESP32Servo, 50 Hz	%60	iyi
LEDC 50 Hz / 16 bit	%60	iyi
LEDC 50 Hz / 14 bit	%60	iyi
LEDC 250 Hz / 16 bit	%0	kötü
LEDC 400 Hz / 16 bit	%0	kötü
RMT 50 Hz / 1 µs tik	%0	iyi

LEDC ile hem ESC senkronu hem telsiz temizliği aynı anda sağlanamamıştır. RMT ise darbe üretimi için tasarlanmış ayrı bir çevre birimidir; kendi saat bölücüsü vardır ve SPI'ın kullandığı altyapıyı paylaşmaz. Bir tik 1 µs'ye ayarlanır, her motor için tek bir sembol yazılır: darbe genişliği kadar yüksek seviye, periyodun kalanı kadar düşük.

RMT darbe üretimi • satır 186–236

```

186  NEDEN LEDC DEGIL RMT - hepsi olculdu:
187      LEDC  50 Hz + 16 bit : nRF24 paket kaybı %60
188      LEDC  50 Hz + 14 bit : nRF24 paket kaybı %60
189      LEDC 250 Hz + 16 bit : kayıp yok, ESC'ler senkronsuz
190      LEDC 400 Hz + 16 bit : kayıp yok, ESC'ler senkronsuz
191  ESC'ler 50 Hz istiyor, LEDC 50 Hz'de SPI'i bozuyor - cikmaz.
192
193  RMT (Remote Control) ESP32'nin hassas darbe uretimi icin ayrilmis
194  cevre birimidir; kendi saat bolucusu vardir ve LEDC'nin kullandigi
195  altyapiyi paylasamaz. Servo/ESC darbesi tam olarak bu is icindir.
196
197  Calisma sekli: 1 MHz tik (1 tik = 1 us). Her motor icin tek bir
198  RMT sembolu yazilir:
199      seviye 1, sure = darbe genisligi (1000-2000 us)
200      seviye 0, sure = periyodun geri kalani (20000 - darbe)
201  Sembol her 20 ms'de bir yeniden tetiklenir.                                */
202  const uint32_t PWM_PERIYOT_US = 20000;    // 50 Hz
203  const uint32_t RMT_TIK_HZ      = 1000000; // 1 us cozunurluk
204  const uint32_t MOTOR_YAZ_MS    = 20;      // RMT yeniden tetikleme periyodu
205
206  rmt_data_t motorSembol[4][1];
207  uint16_t  motorHedef[4] = { PWM_MIN, PWM_MIN, PWM_MIN, PWM_MIN };
208
209  bool motorKur() {
210      bool ok = true;

```



```

211     for (int i = 0; i < 4; i++) {
212         if (!rmtInit(MOTOR_PIN[i], RMT_TX_MODE, RMT_MEM_NUM_BLOCKS_1, RMT_TIK_HZ))
213             ok = false;
214     }
215     return ok;
216 }
217
218 /* Hedef degeri kaydeder - fiziksel yazma motorlariGonder() icinde olur */
219 inline void motorYaz(uint8_t i, uint16_t us) {
220     motorHedef[i] = constrain(us, PWM_MIN, PWM_MAX);
221 }
222
223 /* Dort kanalın sembolunu doldurup RMT'ye gonderir. 20 ms'de bir cagrilir.
224     timeout 0 -> engellemeyen (async) gonderim; kontrol dongusu beklemez. */
225 void motorlariGonder() {
226     for (int i = 0; i < 4; i++) {
227         motorSembol[i][0].level0 = 1;
228         motorSembol[i][0].duration0 = motorHedef[i];
229         motorSembol[i][0].level1 = 0;
230         motorSembol[i][0].duration1 = PWM_PERİYOT_US - motorHedef[i];
231         rmtWrite(MOTOR_PIN[i], motorSembol[i], 1, 0);
232     }
233 }
234
235 /* ===== */
236 /* YARDIMCI FONKSİYONLAR */

```

Teşhis yöntemi

Suçlu tahminle değil **elemeye** bulundu. Önce yalnızca telsiz içeren minimal bir test kodu yazıldı — %100 başarı verdi, donanımın sağlam olduğu kesinleşti. Sonra ana koddaki bileşenler tek tek devre dışı bırakıldı. Motor darbe üretimi kapatılınca kayıp sıfırlandı. Bu, hangi katmanın sorumlu olduğunu tek bir ölçümle gösterdi.

4.4 Kazanç yapısı ve jiroskop filtresi

PID kazançları ve jiroskop filtresi · satır 119–145

```

119 struct Kazanc { float kp, ki, kd; };
120
121 //           Kp      Ki      Kd
122 Kazanc G_ROLL = { 1.20f, 0.00f, 3.00f }; // filtre sayesinde Kd tutulabiliyor
123 Kazanc G_PITCH = { 1.20f, 0.00f, 3.00f }; // roll ile aynı (govde simetrik)
124 Kazanc G_YAW = { 0.50f, 0.00f, 0.00f }; // hiz modu - yalnızca Kp
125
126 /* ----- JIROSKOP ALCAK GECIRGEN FİLTRESİ ---
127     Pervane titreşimi jiroskopa yüksek frekanslı gürültü olarak gelir.
128     D terimi bunu gerçek hareket sanıp motorlara yansıtır, motorlar daha
129     çok titrer - kendini besleyen bir döngü. OLCULDU: orta gazda belirgin.
130
131     Birinci derece (ustel) filtre:
132         suzulmus = suzulmus + ALFA * (ham - suzulmus)
133     ALFA 0 ile 1 arasındadır:
134         1.00 -> filtre yok, ham veri
135         0.20 -> güçlü yumusatma (~9 Hz kesme, 250 Hz örneklemede)
136         0.10 -> çok güçlü (~4 Hz kesme) - gecikme artar
137
138     TAKAS: filtre gecikme getirir. Çok düşük ALFA ile kontrolcü gec
139     tepki verir ve araç sallanmaya başlar. Önce 0.20 dene; titreşim
140     surerse 0.12'ye in, tepki tembellesirse 0.30'a cik.
141
142     NOT: Bu bir YAZILIM çözümüdür, mekanik titreşimi ORTADAN KALDIRMAZ,
143     yalnızca kontrolcuya ulaşmasını engeller. Dengesiz pervane veya
144     gevsek montaj varsa asıl çözüm orada. */
145 const float GYRO_ALFA = 0.20f;

```

Kazançlar bilinçli olarak sıfırdır

Modül, kazançlar sıfırken güvenli biçimde çalışır: motorlar yalnızca gaz komutunu izler, düzeltme uygulanmaz. Ayar prosedürü **SDD-QC-001 Bölüm 8.3**'te tanımlıdır ve test standı üzerinde, pervaneler takılı olarak yapılır. Yorum satırlarındaki değerler başlangıç noktası önerisidir, doğrulanmış değer değildir.

4.5 Veri yapıları ve durum değişkenleri

veri yapıları · satır 163–184

```

163 struct Paket {                      // telsiz üzerinden gelen cerceve (10 bayt)
164     uint16_t kanal[4];              // 0:gaz 1:yaw 2:pitch 3:roll (1000-2000)
165     uint16_t sayac;                 // paket sirasi - kayip tespiti icin
166 };
167 Paket gelen;
168
169 uint16_t kanal[4] = { 1000, 1500, 1500, 1500 }; // guvenli baslangic
170 uint32_t sonPaket = 0;
171 uint16_t sonSayac = 0;
172 uint32_t kayipPaket = 0;
173 bool     baglanti = false;
174 bool     armed    = false;
175 uint32_t donguSon = 0;
176 float    dt       = 1.0f / DONGU_HZ;
177
178 struct PidDurum {
179     float integral;                // birikmis integral terimi (us)
180 };
181 PidDurum dRoll = { 0.0f };
182 PidDurum dPitch = { 0.0f };
183 PidDurum dYaw = { 0.0f };
184

```

Paket yapısı verici tarafındakiyle birebir aynıdır. Alan sırası veya tipi bir tarafta değişirse veri kayar ve anlamsız değerler okunur — bu, doğrudan bayt kopyalamaya dayanan protokollerin bilinen maliyetidir ve iki dosyanın birlikte sürümlemesini gerektirir.

4.6 Yardımcı fonksiyonlar

kanalKomut() ve pidHesapla() · satır 186–236

```

186 NEDEN LEDC DEGIL RMT - hepsi olculdu:
187     LEDC 50 Hz + 16 bit : nRF24 paket kaybi %60
188     LEDC 50 Hz + 14 bit : nRF24 paket kaybi %60
189     LEDC 250 Hz + 16 bit : kayip yok, ESC'ler senkronsuz
190     LEDC 400 Hz + 16 bit : kayip yok, ESC'ler senkronsuz
191     ESC'ler 50 Hz istiyor, LEDC 50 Hz'de SPI'i bozuyor - cikmaz.
192
193     RMT (Remote Control) ESP32'nin hassas darbe uretimi icin ayrilmis
194     cevre birimidir; kendi saat bolucusu vardir ve LEDC'nin kullandigi
195     altyapiyi paylasamaz. Servo/ESC darbesi tam olarak bu is icindir.
196
197     Calisma sekli: 1 MHz tik (1 tik = 1 us). Her motor icin tek bir
198     RMT sembolu yazilir:
199         seviye 1, sure = darbe genisligi (1000-2000 us)
200         seviye 0, sure = periyodun geri kalani (20000 - darbe)
201     Sembol her 20 ms'de bir yeniden tetiklenir.                                */
202 const uint32_t PWM_PERIYOT_US = 20000; // 50 Hz
203 const uint32_t RMT_TIK_HZ     = 1000000; // 1 us cozunurluk
204 const uint32_t MOTOR_YAZ_MS   = 20;     // RMT yeniden tetikleme periyodu
205
206 rmt_data_t motorSembol[4][1];
207 uint16_t    motorHedef[4] = { PWM_MIN, PWM_MIN, PWM_MIN, PWM_MIN };
208

```

```

209 bool motorKur() {
210     bool ok = true;
211     for (int i = 0; i < 4; i++) {
212         if (!rmtInit(MOTOR_PIN[i], RMT_TX_MODE, RMT_MEM_NUM_BLOCKS_1, RMT_TIK_HZ))
213             ok = false;
214     }
215     return ok;
216 }
217
218 /* Hedef degeri kaydeder - fiziksel yazma motorlariGonder() icinde olur */
219 inline void motorYaz(uint8_t i, uint16_t us) {
220     motorHedef[i] = constrain(us, PWM_MIN, PWM_MAX);
221 }
222
223 /* Dort kanalın sembolunu doldurup RMT'ye gonderir. 20 ms'de bir cagrilir.
224     timeout 0 -> engellemeyen (async) gonderim; kontrol dongusu beklemez. */
225 void motorlariGonder() {
226     for (int i = 0; i < 4; i++) {
227         motorSembol[i][0].level0 = 1;
228         motorSembol[i][0].duration0 = motorHedef[i];
229         motorSembol[i][0].level1 = 0;
230         motorSembol[i][0].duration1 = PWM_PERIYOT_US - motorHedef[i];
231         rmtWrite(MOTOR_PIN[i], motorSembol[i], 1, 0);
232     }
233 }
234
235 /* ===== */
236 /* YARDIMCI FONKSIYONLAR */

```

Türev terimi hesaplanmaz; jiroskoptan okunur. İlk gerçeekte türev, açının sayısal farkıydı. Kontrol döngüsü 250 Hz'de çalıştığı için $dt = 4$ ms'dir ve sensördeki 0.1° 'lik gürültü $25^\circ/s$ gibi görünür; $K_d = 8$ ile çarpıldığında $200 \mu s$ 'lik sahte bir düzeltme üretir. Ölçüldüğü üzere araç hareketsizken motorlar 1100-1438 arasında savruluyordu.

MPU6050 açısal hızı zaten doğrudan ölçtüğü için türev almaya gerek yoktur. Bölme işlemi ortadan kalkınca gürültü büyütmesi de kalkar. Gerçek uçuş kontrolcülerinde D terimini bu şekilde kurar. Jiroskop çıkışı ayrıca birinci derece alçak geçiren filtreden ($\alpha = 0.20$) geçirilerek pervane titreşimi bastırılır — filtre olmadan titreşim D terimi üzerinden motorlara geri beslenir ve kendini besleyen bir döngü oluşur.

Integral windup koruması iki katmanlıdır: birikim yalnızca bisikletle bayrağı doğruyken yapılır (silahlı ve gaz eşğin üstünde), ayrıca birikmiş değer $\pm 120 \mu s$ ile sınırlanır. Bu olmadan, araç yerde beklerken integral terimi sürekli birikir ve kalkış anında ani bir yatışa yol açardı (H-09).

4.7 Başlatma dizisi

setup() · satır 290–341

```

290 void setup() {
291     Serial.begin(115200);
292     delay(300);
293     Serial.println(F("\n=== ESP32 UCUS KONTROL v6.0 ==="));
294
295     /* --- 1. ADIM: motorlar önce susturulur -----
296         ESC'ler enerji aldığı anda geçerli bir düşük sinyal görmezse hata bipi
297         çalar veya beklenmedik şekilde durabilir. Bu yüzden ilk iş motor
298         çıkışlarını PWM_MIN'e sabitlemektir. */
299     motorKur();
300     motorlariYaz(PWM_MIN);
301     Serial.println(F("[OK] Motor çıkışları PWM_MIN'e sabitlendi (LEDC)"));
302
303     /* --- 2. ADIM: atalet ölçüm birimi ----- */
304     Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
305     Wire.setClock(400000); // hızlı I2C - dongu butcesi için
306     byte durum = mpu.begin();

```

```

307     if (durum != 0) {
308         Serial.print(F("[HATA] MPU6050 bulunamadi, kod: "));
309         Serial.println(durum);
310         while (1) { motorlariYaz(PWM_MIN); delay(200); } // guvenli kilit
311     }
312     Serial.println(F("[..] Kart DUZ ve HAREKETSIZ dursun, kalibre ediliyor"));
313     delay(1200);
314     mpu.calcOffsets(); // gyro + ivme sapma giderme
315     Serial.println(F("[OK] MPU6050 hazir"));
316
317     /* --- 3. ADIM: telsiz ----- */
318     if (!radio.begin()) {
319         Serial.println(F("[HATA] nRF24 bulunamadi"));
320         while (1) { motorlariYaz(PWM_MIN); delay(200); }
321     }
322     radio.openReadingPipe(0, ADRES);
323     radio.setPALevel(RF_GUC); // yukaridaki sabitten
324     /* VERI HIZI - ALICI VE VERICI AYNI OLMALI
325        250 kbps menzil icin en iyisi, ANCAK piyasadaki nRF24 modullerinin
326        buyuk kısmi Nordic degil Si24R1 klonudur ve klonlarda 250 kbps modu
327        yarı yarıya paket kaybı verir. printPrettyDetails() klonu ayırt
328        edemez - ikisi de "nRF24L01+" görünür.
329        Kayıp yuksekse once bu satiri RF24_1MBPS yapip test et. */
330     radio.setDataRate(RF24_1MBPS); // klon uyumluluğu için
331     radio.setChannel(RF_KANAL);
332     radio.startListening();
333     Serial.println(F("[OK] nRF24 dinlemede"));
334
335     sonPaket = millis();
336     donguSon = micros();
337     Serial.println(F("=== HAZIR - arming için gaz kolunu en alta cekin ===\n"));
338 }
339
340 /* ===== */
341 /* ANA DONGU - sabit 250 Hz */

```

Başlatma sırası emniyet gerekçesiyle sabittir. **İlk iş motor çıkışlarını asgari değere sabitlemektir**; ESC'ler enerji aldıklarında geçerli bir düşük sinyal görmezlerse hata durumuna geçer veya beklenmedik biçimde dönebilir. Ancak bundan sonra sensör ve telsiz başlatılır.

IMU veya telsiz başlatılamazsa yazılım sonsuz döngüye girer ve bu döngü içinde motorları asgari değerde tutmayı sürdürür. Bu, sessizce hatalı çalışmaktansa **güvenli biçimde durmayı** tercih eden bir tasarımdır (SYS-S-05).

4.8 Ana döngü — alım, failsafe ve arming

loop() – alım, failsafe, arming · satır 345–392

```

345     /* ----- 1. TELSİZ: gelen cerceveyi al ve dogrula ----- */
346     if (radio.available()) {
347         radio.read(&gelen, sizeof(gelen));
348
349         // Aralik disi deger = bozuk cerceve. Bu filtre olmadan tek bir hatali
350         // paket motorlara sicrama komutu olarak gidebilir.
351         bool gecerli = true;
352         for (int i = 0; i < 4; i++)
353             if (gelen.kanal[i] < 950 || gelen.kanal[i] > 2050) gecerli = false;
354
355         if (gecerli) {
356             // paket kaybı istatistigi (telemetry / menzil analizi için)
357             if (sonSayac != 0 && (uint16_t)(gelen.sayac - sonSayac) > 1)
358                 kayipPaket += (uint16_t)(gelen.sayac - sonSayac) - 1;
359             sonSayac = gelen.sayac;
360
361             for (int i = 0; i < 4; i++) kanal[i] = gelen.kanal[i];

```

```

362     sonPaket = millis();
363     baglanti = true;
364 }
365 }
366
367 /* ----- 2. FAILSAFE: baglanti kaybi ----- */
368 if (millis() - sonPaket > BAGLANTI_ZAMAN) {
369     if (baglanti) Serial.println(F(">>> FAILSAFE - baglanti kesildi"));
370     baglanti = false;
371     armed = false;           // silahsızlan
372     kanal[0] = PWM_MIN;      // gaz kesildi
373     kanal[1] = kanal[2] = kanal[3] = 1500;
374     pidSifirla();
375     motorlariYaz(PWM_MIN);
376 }
377
378 /* ----- 3. ARMING MANTIGI ----- */
379 Silahlanma sarti: baglanti VAR + gaz kolu en altta.
380 Silahsızlanma : gaz kolu esigin altina inince.
381 Bu, kod resetlendiginde veya kumanda yeniden baglandiginda motorların
382 kendiliginden calismasini engelleyen birincil emniyet katmanidir. */
383 if (!armed && baglanti && kanal[0] <= GAZ_ESIK) {
384     armed = true;
385     pidSifirla();
386     Serial.println(F(">>> ARMED - motorlar aktif"));
387 }
388 if (armed && kanal[0] <= GAZ_ESIK) {
389     // yerde bekleme: PID birikmesin
390     pidSifirla();
391 }
392 }

```

Emniyet blokları zamanlama kapısının **öncesinde** yer alır. Bunun anlamı, failsafe ve arming kontrollerinin her turda — periyot dolmasa bile — çalışmasıdır. Kontrol yasası 4 ms'de bir güncellenir, ancak bağlantı kaybı algılaması mümkün olan en kısa sürede yapılır.

Failsafe tetiklendiğinde yalnızca gaz kesilmez; sistem aynı zamanda armed bayrağını düşürür ve PID durumlarını sıfırlar. Bağlantı geri geldiğinde motorlar kendiliğinden çalışmaz — pilotun silahlanma dizisini yeniden uygulaması gerekir.

4.9 Ana döngü — kestirim, PID ve motor karıştırma

loop() – kestirim, PID, karıştırma · satır 393–443

```

393 /* ----- 4. SABIT DONGU ZAMANLAMASI ----- */
394 PID turev ve integral terimleri sabit dt varsayar. Dongu suresi
395 degiskense kazanclar da fiilen degisir ve ayar tutmaz. */
396 uint32_t simdi = micros();
397 if (simdi - donguSon < DONGU_US) return;
398 dt = (simdi - donguSon) / 1000000.0f;
399 donguSon = simdi;
400
401 /* ----- 5. ATALET OLCUMU ----- */
402 mpu.update();
403 float rollOlculen = mpu.getAngleX(); // derece
404 float pitchOlculen = mpu.getAngleY(); // derece
405
406 /* Jiroskop okumalari suzultur - ham degerler pervane titresimi tasir */
407 static float rollHizi = 0.0f, pitchHizi = 0.0f, yawHizi = 0.0f;
408 rollHizi += GYRO_ALFA * (mpu.getGyroX() - rollHizi);
409 pitchHizi += GYRO_ALFA * (mpu.getGyroY() - pitchHizi);
410 yawHizi += GYRO_ALFA * (mpu.getGyroZ() - yawHizi);
411
412 /* ----- 6. KUMANDA KOMUTLARI ----- */
413 float rollHedef = kanalKomut(kanal[3], MAX_ACI);

```

```

414 float pitchHedef = kanalKomut(kanal[2], MAX_ACI);
415 float yawHedef   = kanalKomut(kanal[1], MAX_YAW_HIZ);
416 uint16_t gaz     = constrain(kanal[0], PWM_MIN, PWM_TAVAN);
417
418 /* ----- 7. PID ----- */
419 Roll ve pitch : ACI modu - hedef aci, olculen aci
420 Yaw           : HIZ modu - hedef donus hizi, olculen gyro hizi
421 Yaw'da aci kullanilmaz cunku manyetometre olmadan yaw acisi surekli
422 kayar (drift). Donus hizi ise driftten etkilenmez. */
423 bool integralAktif = armed && (gaz > GAZ_ESIK + 60);
424
425 float uRoll = pidHesapla(dRoll, G_ROLL, rollHedef, rollOlculen,
426                          rollHizi, integralAktif);
427 float uPitch = pidHesapla(dPitch, G_PITCH, pitchHedef, pitchOlculen,
428                           pitchHizi, integralAktif);
429 float uYaw = pidHesapla(dYaw, G_YAW, yawHedef, yawHizi,
430                        0.0f, integralAktif);
431
432 /* ----- 8. MOTOR KARISTIRMA (X konfigurasyonu) ----- */
433 Isaret mantigi:
434 roll olculen > hedef (saga fazla yatik) -> sag motorlar artar
435 pitch olculen > hedef (burun fazla yukari) -> on motorlar artar
436 yaw saga donus komutu -> CCW motorlar artar (tepki torku)
437 M1 on-sag CW | M2 arka-sag CCW | M3 arka-sol CW | M4 on-sol CCW */
438 int m[4];
439 m[0] = gaz - uRoll - uPitch - uYaw; // M1 on-sag (CW)
440 m[1] = gaz - uRoll + uPitch + uYaw; // M2 arka-sag (CCW)
441 m[2] = gaz + uRoll + uPitch - uYaw; // M3 arka-sol (CW)
442 m[3] = gaz + uRoll - uPitch + uYaw; // M4 on-sol (CCW)
443

```

Yaw eksenini için `getAngleZ()` değil `getGyroZ()` kullanılır. Gerekçe SDD-QC-001 Bölüm 3.3'te ayrıntılıdır: manyetometre yokluğunda yaw açısı sürekli sürüklenir, ancak açısal hız ölçümü bu sürüklenmeden etkilenmez. Kontrol değişkeninin doğru seçilmesi, donanım eksikliğini yazılım mimarisiyle telafi etmenin örneğidir.

Motor karıştırma bloğundaki işaretler, motorun gövde üzerindeki konumundan ve dönüş yönünden türetilir. Yaw düzeltilmesi, saat yönünün tersine dönen motorlarla saat yönünde dönenler arasındaki tepki torku farkı üzerinden üretilir; bu nedenle işaretler çapraz motorlarda eşleşir.

4.10 Çıkış, RMT gönderim ve telemetri

loop() – çıkış, RMT gönderim, telemetri · satır 444–482

```

444 /* ----- 9. CIKIS SINIRLAMA VE YAZMA ----- */
445 if (!armed || gaz <= GAZ_ESIK) {
446     // Silahsız veya gaz kapalı: PID ne derse desin motorlar durur.
447     // Bu, arming kontrolünden BAGIMSIZ ikinci bir emniyet kapisidir.
448     motorlariYaz(PWM_MIN);
449     for (int i = 0; i < 4; i++) m[i] = PWM_MIN; // telemetri dogru gstersin
450 } else {
451     for (int i = 0; i < 4; i++) {
452         m[i] = constrain(m[i], PWM_ARM, PWM_MAX);
453         motorYaz(i, (uint16_t)m[i]);
454     }
455 }
456
457 /* ----- 10. RMT GONDERIM (50 Hz) ----- */
458 PID 250 Hz'de hesaplanır ama ESC'ye giden darbe 50 Hz'dir. Ara
459 hesaplar motorHedef[] icinde birikir, en guncel deger gönderilir.
460 Bu blok OLMAZSA RMT yalnızca setup'ta bir kez tetiklenir ve
461 motorlar hiçbir komuta tepki vermez. */
462 static uint32_t rmtSon = 0;
463 if (millis() - rmtSon >= MOTOR_YAZ_MS) {

```

```
464     rmtSon = millis();
465     motorlariGonder();
466 }
467
468 /* ----- 11. TELEMETRI (10 Hz) ----- */
469 static uint32_t yaz = 0;
470 if (millis() - yaz > 100) {
471     yaz = millis();
472     Serial.print(baglanti ? F("OK ") : F("KOP "));
473     Serial.print(armed ? F("ARM ") : F("--- "));
474     Serial.print(F("T:")); Serial.print(kanal[0]);
475     Serial.print(F(" R:")); Serial.print(roll0lculen, 1);
476     Serial.print(F(" P:")); Serial.print(pitch0lculen, 1);
477     Serial.print(F(" Yr:")); Serial.print(yawHizi, 1);
478     Serial.print(F(" | M "));
479     for (int i = 0; i < 4; i++) { Serial.print(m[i]); Serial.print(' '); }
480     Serial.print(F("| kayip:")); Serial.println(kayipPaket);
481 }
482 }
```

Son emniyet kapısı buradadır: silahlı değilse veya gaz kapalıysa, PID'nin ürettiği değer ne olursa olsun motorlara asgari komut yazılır. Bu, kontrol yasasındaki olası bir hatanın motorları çalıştırmasını engelleyen bağımsız bir katmandır — savunma yazılımlarındaki *defence in depth* ilkesinin küçük ölçekli uygulaması.

Telemetri çıktısı 10 Hz'e seyreltilmiştir. Seri port yazma işlemi görece pahalıdır (~800 µs); her döngüde çalıştırılıyorsa 4 ms'lik bütçenin beşte birini tek başına tüketirdi.

5. Araç Yazılımları

Araç yazılımları, uçuş yazılımına geçmeden önce donanımı doğrulamak için yazılmıştır. Küçük olmaları aldaticıdır — her ikisi de geri döndürülmesi pahalı hataları önler.

5.1 ESC uç nokta kalibrasyonu

Kalibrasyon yapılmazsa her ESC kendi fabrika gaz aralığını kullanır. Aynı PWM değeri farklı motorlarda farklı devir üretir; araç havada sürekli bir yöne kayar ve bu, PID ayarıyla düzeltilemeyecek bir asimetridir.

arac_esc_kalibrasyon.ino · satır 19–66

```

19  /* Motor pinleri - UCUS KODUYLA AYNI OLMALI */
20  #define PIN_M1  25    // on-sag , CW
21  #define PIN_M2  26    // arka-sag, CCW
22  #define PIN_M3  32    // arka-sol, CW
23  #define PIN_M4  33    // on-sol , CCW
24
25  const uint8_t MOTOR_PIN[4] = { PIN_M1, PIN_M2, PIN_M3, PIN_M4 };
26
27  const uint16_t PWM_MIN      = 1000;
28  const uint16_t PWM_MAX      = 2000;
29  const uint32_t PWM_PERİYOT_US = 20000;    // 50 Hz
30  const uint32_t RMT_TIK_HZ    = 1000000;    // 1 us cozunurluk
31
32  rmt_data_t motorSembol[4][1];
33  uint16_t motorHedef[4] = { PWM_MIN, PWM_MIN, PWM_MIN, PWM_MIN };
34
35  bool motorKur() {
36      bool ok = true;
37      for (int i = 0; i < 4; i++)
38          if (!rmtInit(MOTOR_PIN[i], RMT_TX_MODE, RMT_MEM_NUM_BLOCKS_1, RMT_TIK_HZ))
39              ok = false;
40      return ok;
41  }
42
43  void motorlariGonder() {
44      for (int i = 0; i < 4; i++) {
45          motorSembol[i][0].level0 = 1;
46          motorSembol[i][0].duration0 = motorHedef[i];
47          motorSembol[i][0].level1 = 0;
48          motorSembol[i][0].duration1 = PWM_PERİYOT_US - motorHedef[i];
49          rmtWrite(MOTOR_PIN[i], motorSembol[i], 1, 0);
50      }
51  }
52
53  void hepsineAyarla(uint16_t us) {
54      for (int i = 0; i < 4; i++) motorHedef[i] = constrain(us, PWM_MIN, PWM_MAX);
55  }
56
57  /* Belirtilen sure boyunca 50 Hz'de darbe uretmeyi surdurur.
58   ESC sinyali kesilmesini ariza sayar, bu yuzden bekleme sirasinda da
59   gonderim devam etmelidir - duz delay() KULLANILAMAZ. */
60  void bekleVeGonder(uint32_t sure_ms) {
61      uint32_t bas = millis();
62      while (millis() - bas < sure_ms) {
63          motorlariGonder();
64          delay(20);
65      }
66  }

```


loop() neden boş değil

İlk sürümde loop() boş bırakılmıştı. Kalibrasyon bittiğinde sinyal üretimi durdu ve ESC'ler sürekli hata bipi çalmaya başladı — sinyal kaybı, ESC için bir arıza durumudur. Düzeltme olarak döngü, asgari sinyali sürdürmeye devam eder. Küçük bir ayrıntı gibi görünse de, ESC'nin sinyal beklentisinin sürekli olduğunu anlamayı gerektirir.

5.2 Motor dönüş yönü testi

X konfigürasyonunda çapraz motorlar aynı yöne, komşu motorlar ters yöne dönmelidir. Bu düzen sağlanmazsa dört motorun tepki torku toplamı sıfırlanmaz ve araç yaw ekseninde kontrolsüz döner.

arac_motor_test.ino · satır 19–59

```

19  #include <ESP32Servo.h>
20
21  Servo esc[4];
22  const int PIN[4] = { 12, 13, 14, 27 };
23  const uint16_t TEST_GAZ = 1150;    // rolanti uzeri - yon gormeye yeter
24
25  void setup() {
26    Serial.begin(115200);
27    delay(500);
28
29    for (int i = 0; i < 4; i++) {
30      esc[i].setPeriodHertz(50);
31      esc[i].attach(PIN[i], 1000, 2000);
32      esc[i].writeMicroseconds(1000);
33    }
34
35    Serial.println(F("=== MOTOR TESTI ==="));
36    Serial.println(F("1-4 : o motoru 3 saniye dondur"));
37    Serial.println(F("0   : hepsini durdur"));
38    Serial.println(F("PERVANE TAKILI OLMAYACAK"));
39  }
40
41  void loop() {
42    if (!Serial.available()) return;
43    char c = Serial.read();
44
45    if (c >= '1' && c <= '4') {
46      int m = c - '1';
47      Serial.print(F(">> Motor ")); Serial.print(m + 1);
48      Serial.println(F(" donuyor (3 sn)"));
49      esc[m].writeMicroseconds(TEST_GAZ);
50      delay(3000);
51      esc[m].writeMicroseconds(1000);
52      Serial.println(F("   durdu"));
53    }
54
55    if (c == '0') {
56      for (int i = 0; i < 4; i++) esc[i].writeMicroseconds(1000);
57      Serial.println(F(">> HEPSI DURDU"));
58    }
59  }

```

Test kodu, motorları **tek tek** çalıştırır. Bu, tanı koymayı mümkün kılan tasarım tercihidir: dördü aynı anda çalıştırılıyorsa yanlış dönen motoru ayırt etmek zorlaşır. Devreye alma sırasında her motor önce geçici bağlantıyla test edilmiş, dönüş yönü doğrulandıktan sonra kalıcı olarak lehimlenmiştir — geri döndürülemez adımın doğrulamadan sonraya bırakılması ilkesi.

6. Emniyet Mekanizmalarının Özeti

Emniyet, tek bir mekanizmaya değil birbirinden bağımsız katmanlara dayanır. Aşağıdaki tablo, her katmanın hangi kod bloğunda gerçekleştiğini ve hangi arıza modunu (SDD-QC-001 Bölüm 6) karşıladığını gösterir.

#	Mekanizma	Kod konumu	Karşıladığı risk
1	Başlangıçta motor çıkışlarının asgariye sabitlenmesi	setup(), 1. adım	H-03
11	Jiroskop alçak geçirgen filtresi (titreşim yalıtımı)	loop(), 5. blok	H-11
2	IMU / telsiz başlatma hatasında güvenli kilit	setup(), 2.-3. adım	H-06
3	Çerçeve aralık süzgeci (950–2050 μ s)	loop(), 1. blok	H-02
4	500 ms zaman aşımli failsafe	loop(), 2. blok	H-01
5	Failsafe'te otomatik silahsızlanma	loop(), 2. blok	H-01
6	Arming zorunluluğu (gaz asgari şartı)	loop(), 3. blok	H-03
7	Koşullu integral birikimi + windup sınırı	pidHesapla()	H-09
8	Eksen başına PID çıkış sınırlaması (± 320 μ s)	pidHesapla()	—
9	Gaz tavanı (1900 μ s) ile düzeltme baş payı	loop(), 6. blok	—
10	Silahsız / gaz kapalı durumunda koşulsuz asgari çıkış	loop(), 9. blok	H-03

Tasarım ilkesi

10 numaralı mekanizma bilinçli bir fazlalıktır: 6 numaralı arming kontrolü zaten aynı işi yapar. Ancak emniyet-kritik yollarda tek bir kontrole güvenmek, o kontroldeki bir mantık hatasının doğrudan tehlikeye dönüşmesi anlamına gelir. Bağımsız iki kapı, birinin hatalı olması hâlinde diğerinin tutmasını sağlar.

7. Zamanlama Bütçesi ve Kaynak Kullanımı

Kontrol döngüsünün 4 ms'lik periyodu içinde yapılan işlerin süreleri aşağıdadır. Ölçümler mikrosaniye sayacıyla alınmış yaklaşık değerlerdir.

İşlem	Süre	Bütçe payı	Not
nRF24 çerçeve okuma (SPI)	~120 µs	%3.0	Yalnızca çerçeve geldiğinde
MPU6050 okuma + açış hesabı (I ² C)	~900 µs	%22.5	En büyük tekil kalem
PID hesabı (3 eksen)	~40 µs	%1.0	Kayan nokta birimi kullanılıyor
Motor karıştırma + sınırlama	< 20 µs	%0.5	Tamsayı aritmetiği
PWM yazma (4 kanal, LEDC)	~30 µs	%0.8	Donanım hızlandırılmalı
Telemetri (10 Hz seyriltilmiş)	~800 µs	%2.0	Etkin pay: 800/10 turda
Toplam (tipik)	≈ 1.2 ms	%30	Yaklaşık %70 rezerv

I²C hızı 400 kHz'e çıkarılarak IMU okuma süresi yaklaşık yarıya indirilmiştir; varsayılan 100 kHz ile bu kalem tek başına bütçenin yarısına yaklaşıyordu. Kalan rezerv, ileride eklenecek irtifa tutma, telemetri geri kanalı veya kara kutu kaydı gibi işlevler için bilinçli olarak ayrılmıştır.

7.1 Çift çekirdek kullanımı

Mevcut sürüm tek çekirdek üzerinde çalışır. ESP32'nin ikinci çekirdeği, ileride telemetri ve haberleşme görevlerine ayrılarak kontrol döngüsünün determinizmi daha da güçlendirilebilir. Bu, FreeRTOS görev ayrımı gerektiren orta vadeli bir geliştirmedir.

8. Derleme, Bağımlılıklar ve Devreye Alma

8.1 Araç zinciri

Bileşen	Sürüm / ayar	Not
Arduino IDE	2.x	—
ESP32 kart paketi	esp32 by Espressif Systems 3.3.x	Boards Manager üzerinden
Kart seçimi	ESP32 Dev Module	—
Yükleme hızı	115200	921600'de seri gürültü hatası gözlemlendi
USB köprü sürücüsü	Silicon Labs CP210x VCP	Sürücü kurulmadan port görünmez
Seri monitör	115200 baud	—

8.2 Kütüphane bağımlılıkları

Kütüphane	Yazar	Kullanan modül	İşlev
RF24	TMRh20	verici, alıcı	nRF24L01+ sürücüsü
MPU6050_light	rfetick	alıcı	IMU okuma ve açı kestirimi
—	—	—	ESP32Servo kullanılmıyor ; darbe üretimi RMT ile doğrudan
Wire	çekirdek	alıcı	I ² C
SPI	çekirdek	verici, alıcı	SPI

Kütüphane seçiminde isim benzerliğine dikkat edilmelidir: RF24 adıyla birden fazla paket vardır ve yalnızca TMRh20 sürümü bu kodla uyumludur. Aynı biçimde Servo kütüphanesi AVR mimarisi içindir, ESP32'de derlenmez.

8.3 Devreye alma sırası

- Batarya çıkarılır.** Kod yükleme sırasında ESC'nin BEC hattı ile USB beslemesi çakışır ve seri iletişim bozulur (D-02).
- arac_esc_kalibrasyon yüklenir, seri monitör açılır.
- "MAX sinyal aktif" mesajı görüldüğünde batarya takılır; bip dizisi beklenir.
- arac_motor_test yüklenir; motorlar tek tek çalıştırılıp dönüş yönleri doğrulanır.
- Yönler doğrulandıktan sonra motor kabloları kalıcı olarak lehimlenir.
- verici_v2_kumanda verici karta, alıcı_v3_ucus_kontrol alıcı karta yüklenir.
- Kazançlar sıfırken bağlantı, arming ve failsafe davranışı doğrulanır — **pervanesiz**.
- Test standına geçilir; işaret doğrulaması ve kazanç ayarı yapılır.

Kalıcı kural

Her kod yüklemesinden önce batarya çıkarılır. Bu, hem seri iletişim kararlılığı hem de yükleme sırasında motorların beklenmedik biçimde çalışmaması için gereklidir.

9. Sürüm Geçmişi

Sürüm	Modül	Değişiklik	Gerekçe
v1.0	verici	Temel 4 kanal aktarımı, doğrusal ölçekleme	İlk çalışan haberleşme
v1.1	verici	Merkez kalibrasyonu ve ± 60 ölü bölge eklendi	Ham ADC merkezi 2048 değil
v2.0	verici	Kilitli gaz + karesel tepki eğrisi	Paylaşımlı merkezleme yayı kısıtı (D-01)
v2.1	verici	Ölü bölge $\pm 80^\circ$ 'e genişletildi	Yaw kanalında artık titreşim ölçüldü
v1.0	alıcı	Telsiz alımı ve seri çıktı	Bağlantı doğrulaması
v1.1	alıcı	Aralık süzgeci ve failsafe eklendi	Boş tampon okuma arızası (D-03)
v2.0	alıcı	MPU6050 entegrasyonu, açılı çıktı	Kestirim katmanı
v3.0	alıcı	Arming, PID, motor karıştırma, ESC çıkışı, telemetri	Tam uçuş kontrol yazılımı
v3.1	alıcı	Türev sıçraması düzeltildi (ilkTur bayrağı)	Arming anında motorlara darbe gidiyordu
v3.2	alıcı	Yaw kazancı 2.00 $\rightarrow$ 0.80	Elle çevirmede motorlar tavana dayandı
v4.0	alıcı	ESP32Servo kaldırıldı, LEDC'ye geçildi	Paket kaybı araştırması
v4.1	alıcı	Motor pinleri 12/13/14/27 $\rightarrow$ 25/26/32/33	Strapping ve HSPI pinlerinden kaçınma
v4.2-4.4	alıcı	PWM frekans/çözünürlük denemeleri	ESC senkronu ile telsiz temizliği çakışması
v5.0	alıcı	LEDC bırakıldı, RMT çevre birimine geçildi	Paket kaybı sıfırlandı
v5.1	alıcı	Türev jiroskoptan alınmaya başlandı	Sayısal türev sensör gürültüsünü büyütüyordu
v6.0	alıcı	Jiroskop alçak geçirgen filtresi ($\alpha = 0.20$)	Pervane titreşiminin D terimine sızması
v2.2-2.4	verici	RF kanal 115, veri hızı 1 Mbps, PA_MIN	Bağlantı kalitesi araştırması

Çalışan her sürüm ayrı dosya adıyla saklanmıştır. Bu, hatalı bir değişiklikten geri dönüşü mümkün kılan en basit sürüm kontrolü biçimidir ve donanımla çalışırken özellikle değerlidir: yazılım hatası ile donanım arızasını ayırt etmenin en hızlı yolu, bilinen çalışan bir sürüme dönmektir.

Doküman sonu

Sistem düzeyi gereksinimler, mimari kararlar, tehlike analizi ve test kayıtları için **SDD-QC-001 — Sistem Tasarım Dokümanı**'na başvurulmalıdır.